

# Déploiement Docker et Mobile — CareWay

Pour étudiants en 2e année | Version : 0.1.0 | Février 2026  
Niveau requis : bases Docker, ligne de commande Windows/Linux

## Table des matières

- A. Services et images Docker
- B. Explication du Dockerfile et docker-compose.yml
- C. Procédure from scratch — Build, Run, Stop, Reset
- D. Gestion des environnements — Variables .env
- E. Packaging — Archivage et transmission
- F. Mobile — Stratégie APK et préparation

## A. Services et images Docker

### Vue d'ensemble

La stack Docker de CareWay comprend **7 services** définis dans `docker/docker-compose.yml` :

| Service  | Image                            | Port exposé  | Rôle                                       |
|----------|----------------------------------|--------------|--------------------------------------------|
| frontend | Build local (Dockerfile)         | 3000 → 3000  | Application React (Vite dev ou nginx prod) |
| db       | supabase/postgres:15.1.0.147     | 54322 → 5432 | Base de données PostgreSQL                 |
| auth     | supabase/gotrue:v2.99.0          | 9999 → 9999  | Authentification (GoTrue)                  |
| rest     | postgrest/postgrest:v11.2.2      | 54321 → 3000 | API REST auto-générée depuis PostgreSQL    |
| studio   | supabase/studio:20231123-64a766a | 54323 → 3000 | Interface admin Supabase Studio            |
| meta     | supabase/postgres-meta:v0.68.0   | (interne)    | Métadonnées PostgreSQL                     |
| inbucket | inbucket/inbucket:3.0.3          | 54324 → 9000 | Serveur email de test                      |

Les services `db`, `auth`, `rest`, `studio`, `meta`, `inbucket` utilisent le **profil Docker Compose local** — ils ne démarrent que si on utilise `--profile local`.

### URLs d'accès après démarrage

| Service                   | URL                        |
|---------------------------|----------------------------|
| Frontend React            | http://localhost:3000      |
| Supabase Studio (admin)   | http://localhost:54323     |
| API REST (PostgREST)      | http://localhost:54321     |
| Auth (GoTrue)             | http://localhost:9999      |
| PostgreSQL                | localhost:54322 (port TCP) |
| Emails de test (Inbucket) | http://localhost:54324     |

## B. Explication du Dockerfile et docker-compose.yml

### B.1 Dockerfile — `docker/Dockerfile`

Le Dockerfile est **multi-stage** : il définit 4 cibles de build.

```
# Stage 1 : base — installe les dépendances npm
FROM node:20-alpine AS base
WORKDIR /app
COPY package*.json ./
RUN npm install          # ← Note : "npm 0" dans le fichier source = typo, doit
                          # être "npm install"

# Stage 2 : dev — lancement en mode développement avec hot reload
FROM base AS dev
COPY . .
CMD ["npm", "run", "dev", "--", "--host"]
# "--host" expose le serveur Vite sur 0.0.0.0 (accessible depuis l'extérieur du
# conteneur)

# Stage 3 : build — compile l'app pour la production
FROM base AS build
COPY . .
ARG VITE_SUPABASE_URL          # Variables injectées au moment du build
ARG VITE_SUPABASE_ANON_KEY
RUN npm run build              # → génère le dossier /app/dist

# Stage 4 : production — nginx sert les fichiers statiques
FROM nginx:alpine AS production
COPY --from=build /app/dist /usr/share/nginx/html
COPY docker/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

**Note importante :** Le stage `dev` est utilisé par docker-compose (voir `target: dev`). Pour un déploiement production, utiliser `target: production` avec les ARGs appropriés.

### B.2 docker-compose.yml — Détail par service

**Service frontend (toujours actif)**

```

services:
  frontend:
    build:
      context: ..          # Contexte = racine du projet CareWay/
      dockerfile: docker/Dockerfile
      target: dev          # Utilise le stage "dev" (hot reload)
    volumes:
      - ../app             # Monte tout le code source dans /app
      - /app/node_modules  # Exclut node_modules du mount (volume anonyme)
    ports:
      - "3000:3000"        # Accès sur localhost:3000
    environment:
      - CHOKIDAR_USEPOLLING=true  # Nécessaire pour le hot reload sur
Windows/WSL2
      - WATCHPACK_POLLING=true
    env_file:
      - .env               # Charge docker/.env (VITE_SUPABASE_URL, etc.)

```

**Service db (profil local)**

```

db:
  profiles: ["local"]
  image: supabase/postgres:15.1.0.147
  ports:
    - "54322:5432"        # PostgreSQL accessible sur localhost:54322
  environment:
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}  # Mot de passe (défaut:
postgres)
    POSTGRES_DB: postgres
  volumes:
    - db-data:/var/lib/postgresql/data  # Persistance des données
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "postgres"]
    interval: 10s
    timeout: 5s
    retries: 5

```

**Service auth (profil local)**

```

auth:
  profiles: ["local"]
  image: supabase/gotrue:v2.99.0
  environment:
    GOTRUE_JWT_SECRET: ${JWT_SECRET:-super-secret-jwt-...}  # Secret JWT
    GOTRUE_MAILER_AUTOCONFIRM: "true"  # Confirme les emails automatiquement

```

```
(dev)
  GOTRUE_SMTP_HOST: inbucket          # Envoie les emails vers Inbucket
  ports:
    - "9999:9999"
  depends_on:
    db:
      condition: service_healthy      # Attend que PostgreSQL soit prêt
```

## Volume persistant

```
volumes:
  db-data:  # Volume Docker nommé – les données PostgreSQL survivent aux restarts
```

## C. Procédure from scratch

### C.1 Prérequis

| Outil          | Version minimale | Lien de téléchargement                                                                                                  |
|----------------|------------------|-------------------------------------------------------------------------------------------------------------------------|
| Docker Desktop | 24.0+            | <a href="https://www.docker.com/products/docker-desktop">https://www.docker.com/products/docker-desktop</a>             |
| Docker Compose | 2.20+            | Inclus avec Docker Desktop                                                                                              |
| Node.js        | 18+              | <a href="https://nodejs.org/">https://nodejs.org/</a>                                                                   |
| Git            | 2.x              | <a href="https://git-scm.com/">https://git-scm.com/</a>                                                                 |
| WSL2 (Windows) | Windows 10/11    | <a href="https://docs.microsoft.com/fr-fr/windows/wsl/install">https://docs.microsoft.com/fr-fr/windows/wsl/install</a> |

Sur **Windows**, activer WSL2 avant d'installer Docker Desktop.

### C.2 Installation initiale

```
# 1. Cloner le dépôt
git clone https://github.com/NEXUS-AI-Innovation-lab-for-Health/CareWay.git
cd CareWay

# 2. Installer les dépendances Node.js (pour le build local si besoin)
npm install
```

### C.3 Configuration des fichiers d'environnement

```
# Option A : Script automatique (recommandé)
.\setup.ps1

# Option B : Manuel
```

```
# Créer docker/.env avec le contenu :
@"
POSTGRES_PASSWORD=postgres
JWT_SECRET=super-secret-jwt-token-with-at-least-32-characters-long
ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImFub24iLCJleHAiOjE5ODM4MTI5OTZ9.CRXP1A7W0eoJeXxjNni43kdQwgnWNReilDMblyTn_I0
SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImFub24iLCJleHAiOjE5ODM4MTI5OTZ9.CRXP1A7W0eoJeXxjNni43kdQwgnWNReilDMblyTn_I0
VITE_SUPABASE_URL=http://localhost:8000
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImFub24iLCJleHAiOjE5ODM4MTI5OTZ9.CRXP1A7W0eoJeXxjNni43kdQwgnWNReilDMblyTn_I0
"@ | Out-File -FilePath docker/.env -Encoding utf8
```

## C.4 Démarrer la stack

```
# Démarre TOUS les services (frontend + Supabase local complet)
npm run docker:start

# Équivalent exact :
docker compose -f docker/docker-compose.yml --profile local up -d --build
```

Le premier démarrage prend 3-5 minutes (téléchargement des images).  
Les suivants sont instantanés (images en cache).

## C.5 Vérifications après démarrage

```
# Lister les conteneurs en cours
docker ps

# Résultat attendu (7 conteneurs) :
# careway-frontend      Up      0.0.0.0:3000->3000/tcp
# careway-db            Up      0.0.0.0:54322->5432/tcp
# careway-auth          Up      0.0.0.0:9999->9999/tcp
# careway-rest          Up      0.0.0.0:54321->3000/tcp
# careway-studio        Up      0.0.0.0:54323->3000/tcp
# careway-meta          Up
# careway-inbucket      Up      0.0.0.0:54324->9000/tcp

# Vérifier les logs du frontend
docker compose -f docker/docker-compose.yml logs frontend

# Tester les accès navigateur
# → http://localhost:3000      (Frontend React)
# → http://localhost:54323     (Supabase Studio)
```

## C.6 Initialiser la base de données

1. Ouvrir **Supabase Studio** : `http://localhost:54323`
2. Aller dans **SQL Editor**
3. Copier-coller le contenu de `db/schema.sql` et exécuter
4. (Optionnel) Exécuter les scripts dans `src/scripts/` si présents :
  - `src/scripts/init-care-types.sql`
  - `src/scripts/add-patient-medical-fields.sql`
  - `src/demo-data-script.sql` (données de démonstration)

```
# Alternative : exécuter via psql dans le conteneur
docker exec -i careway-db psql -U postgres -d postgres < db/schema.sql
```

## C.7 Arrêter la stack

```
# Arrêt propre (données conservées dans les volumes)
npm run docker:stop

# Équivalent :
docker compose -f docker/docker-compose.yml --profile local down
```

## C.8 Nettoyer complètement (reset total)

```
# ⚠ ATTENTION : supprime tous les volumes et données
npm run docker:clean

# Équivalent :
docker compose -f docker/docker-compose.yml --profile local down -v

# Pour nettoyer aussi les images non utilisées
docker system prune -a
```

## C.9 Reconstruire après modification du code

```
# En mode dev, le hot reload fonctionne automatiquement grâce aux volumes
# Pour forcer un rebuild complet de l'image :
docker compose -f docker/docker-compose.yml build --no-cache frontend
npm run docker:start
```

## C.10 Résolution des problèmes fréquents

**Port 3000 déjà utilisé :**

```
netstat -ano | findstr :3000
taskkill /PID <PID> /F
```

Port 54322 déjà utilisé (PostgreSQL local) :

```
netstat -ano | findstr :54322
taskkill /PID <PID> /F
```

Conteneur frontend ne démarre pas :

```
docker compose -f docker/docker-compose.yml logs frontend
# Vérifier que docker/.env existe et contient VITE_SUPABASE_URL
```

Base de données non accessible :

```
docker exec -it careway-db pg_isready -U postgres
# Si erreur → attendre 30s et réessayer
```

## D. Gestion des environnements

### D.1 Fichiers d'environnement

| Fichier          | Emplacement | Contenu                           | Usage                 |
|------------------|-------------|-----------------------------------|-----------------------|
| docker/.env      | docker/.env | Variables pour docker-compose     | Démarrage Docker      |
| .env             | Racine      | VITE_* pour Vite (Supabase Cloud) | Dev local sans Docker |
| .env.development | Racine      | VITE_* pour Supabase local        | Dev avec Docker       |
| .env.example     | Racine      | Template des variables            | Documentation         |

### D.2 Variables critiques

| Variable               | Valeur locale (Docker)         | Valeur production (Cloud)        |
|------------------------|--------------------------------|----------------------------------|
| VITE_SUPABASE_URL      | http://localhost:8000          | https://<project-id>.supabase.co |
| VITE_SUPABASE_ANON_KEY | Clé JWT demo (voir ci-dessous) | Clé depuis le dashboard Supabase |
| POSTGRES_PASSWORD      | postgres                       | Mot de passe fort                |
| JWT_SECRET             | Valeur demo (≥32 chars)        | Secret fort aléatoire            |

### D.3 Clés demo pour développement local

```
# ANON_KEY (rôle anon) – pour les requêtes publiques
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImFub24iLCJleHAiOiJl50DM4MTI5OTZ9.CRXP1A7W0eoJeXjNni43kdQwgnWNReilDMblyTn_I0

# SERVICE_ROLE_KEY (rôle admin) – ne jamais exposer côté client
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZS1kZW1vIiwicm9sZSI6ImNlc nZpY2Vfcm9sZSI6ImV4CI6MTk4MzgxMjk5Nn0.EGIM96RAZx35lJzdJsyH-qQwv8Hj04zWl196z2-SBc0
```

⚠ Ces clés sont publiques et connues — utilisées uniquement pour les démos locales. En production, générer des clés via le dashboard Supabase.

## D.4 Build production avec variables injectées

```
# Build de l'image production avec les clés Cloud
docker build \
  --target production \
  --build-arg VITE_SUPABASE_URL=https://monprojet.supabase.co \
  --build-arg VITE_SUPABASE_ANON_KEY=ma-clé-anon \
  -t careway-prod \
  -f docker/Dockerfile .
```

## E. Packaging — Archivage et transmission

### E.1 Préparer une archive pour transmission

```
# Se placer à la racine du projet
cd C:\Sae16damwant\CareWay

# Créer une archive ZIP (sans node_modules et volumes)
Compress-Archive -Path . \
  -DestinationPath ..\CareWay-v0.1.0.zip \
  -CompressionLevel Optimal

# Exclure manuellement les dossiers lourds AVANT de zipper
# (supprimer ou déplacer temporairement)
Remove-Item -Recurse -Force node_modules
Remove-Item -Recurse -Force volumes
# Puis zipper, puis réinstaller : npm install
```

### E.2 Fichiers à INCLURE dans l'archive

- ☒ src/
- ☒ db/
- ☒ docker/



- ☒ public/
- ☒ supabase/
- ☒ index.html
- ☒ package.json
- ☒ package-lock.json
- ☒ vite.config.ts
- ☒ tsconfig.json
- ☒ tsconfig.node.json
- ☒ README.md
- ☒ docs/
- ☒ .env.example (template uniquement – PAS le .env réel avec les clés)

### E.3 Fichiers à EXCLUDE (ne jamais transmettre)

- ☒ node\_modules/ → trop lourd (~500 MB), se réinstalle avec `npm install`
- ☒ volumes/ → données Docker locales
- ☒ .env → contient des clés secrètes
- ☒ docker/.env → contient des clés secrètes
- ☒ dist/ → se régénère avec `npm run build`

### E.4 Procédure de réception (côté destinataire)

```
# 1. Décompresser l'archive
Expand-Archive .\CareWay-v0.1.0.zip -DestinationPath .\CareWay

# 2. Se placer dans le projet
cd CareWay

# 3. Installer les dépendances
npm install

# 4. Configurer l'environnement (voir section D)
# Créer docker/.env avec les bonnes valeurs

# 5. Démarrer
npm run docker:start
```

## F. Mobile — Stratégie APK et Préparation

### F.1 État actuel du projet

**CareWay est actuellement une application web uniquement (React SPA).**

Il n'existe pas de code mobile natif dans le dépôt.

La partie mobile est à construire.

### F.2 Approche recommandée — Capacitor (Ionic)

La stratégie la plus adaptée à ce projet est **Capacitor** car :

- Il réutilise le code React existant à 100% (zéro réécriture)
- Il génère un APK Android **et** une IPA iOS depuis le même code
- Il s'intègre directement avec Vite

### F.3 Étapes de préparation à l'APK

#### Étape 1 — Installer Capacitor

```
cd C:\Sae16damwant\CareWay

# Installer les dépendances Capacitor
npm install @capacitor/core @capacitor/cli
npm install @capacitor/android

# Initialiser Capacitor dans le projet
npx cap init CareWay com.nexus.careway --web-dir dist
```

#### Étape 2 — Prérequis Android

| Outil          | Version   | Téléchargement                                                                          |
|----------------|-----------|-----------------------------------------------------------------------------------------|
| Android Studio | Hedgehog+ | <a href="https://developer.android.com/studio">https://developer.android.com/studio</a> |
| JDK            | 17        | Inclus avec Android Studio                                                              |
| Android SDK    | API 34+   | Via Android Studio SDK Manager                                                          |
| Gradle         | Inclus AS | —                                                                                       |

#### Étape 3 — Build de l'application web

```
# Compiler l'app React (génère /dist)
npm run build
# Pour le mobile, les variables d'env doivent pointer vers le Supabase Cloud
# (pas localhost – le mobile ne peut pas résoudre localhost)
```

#### Étape 4 — Ajouter la plateforme Android

```
# Ajouter le projet Android (crée le dossier /android)
npx cap add android

# Synchroniser le build web vers le projet Android
npx cap sync android
```

## Étape 5 — Builder l'APK

```
# Ouvrir Android Studio pour builder
npx cap open android
# Dans Android Studio : Build → Build Bundle(s)/APK(s) → Build APK(s)

# OU en ligne de commande (dans android/)
cd android
.\gradlew assembleDebug      # Windows
./gradlew assembleDebug      # Linux/Mac
```

## Étape 6 — Récupérer l'APK

Chemin de l'APK debug :

```
android/app/build/outputs/apk/debug/app-debug.apk
```

Chemin de l'APK release (signé) :

```
android/app/build/outputs/apk/release/app-release.apk
```

## Étape 7 — Installer sur Android

### Méthode 1 — USB (recommandé pour tests)

```
# Activer "Débogage USB" sur le téléphone
# Connecter via USB
adb install android/app/build/outputs/apk/debug/app-debug.apk
```

### Méthode 2 — Fichier direct

1. Copier `app-debug.apk` sur le téléphone (USB ou email)
2. Ouvrir le fichier depuis le gestionnaire de fichiers Android
3. Autoriser les sources inconnues si demandé

### Méthode 3 — Réseau local

```
# Servir l'APK via HTTP simple
python -m http.server 8080
# Sur le téléphone (même réseau WiFi) : http://192.168.x.x:8080/app-debug.apk
```

## F.4 Points d'attention pour le mobile

| Point | Action requise |
|-------|----------------|
|-------|----------------|

| Point               | Action requise                                                                                            |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| VITE_SUPABASE_URL   | Doit pointer vers Supabase Cloud (https://...), pas localhost                                             |
| Leaflet (cartes)    | Vérifier la compatibilité mobile — ajouter <code>@capacitor/google-maps</code> si besoin                  |
| FranceConnect       | Configurer un redirect URI mobile dans les paramètres FranceConnect                                       |
| CORS Supabase       | Ajouter <code>capacitor://localhost</code> aux origines autorisées                                        |
| Permissions Android | Ajouter dans <code>AndroidManifest.xml</code> : <code>INTERNET</code> , <code>ACCESS_FINE_LOCATION</code> |

## F.5 Fichier `capacitor.config.ts` (à créer)

```
import type { CapacitorConfig } from '@capacitor/cli';

const config: CapacitorConfig = {
  appId: 'com.nexus.careway',
  appName: 'CareWay',
  webDir: 'dist',
  server: {
    // En développement : pointer vers le serveur local
    // url: 'http://10.0.2.2:5173',
    // cleartext: true,
  },
  plugins: {
    SplashScreen: {
      launchShowDuration: 2000,
    },
  },
};

export default config;
```